EA2 – Creación de repositorio: procesamiento funcional de datos para un sistema de gestión de transporte urbano (TecnoMovil Data)

María Julieth Gómez Sarrazola

Programación Orientada a Objetos II

Jorge Armando Julio Cruz

Ingeniería de Software y Datos, IU Digital de Antioquia

Introducción

La Alcaldía de TecnoValle administra un sistema de transporte urbano que mueve más de 450.000 pasajeros al día y genera entre 6 y 12 millones de registros diarios. La plataforma actual, construida con programación imperativa, procesa esos datos de forma secuencial, comparte estructuras mutables y produce errores de conteo por condiciones de carrera. El proyecto TecnoMovil Data busca reemplazarla por una arquitectura modular, declarativa y escalable apoyada en programación funcional.

Este trabajo presenta el módulo funcional de procesamiento y agregación de datos diarios, desarrollado en Java sin frameworks externos. El módulo calcula la afluencia por estación, las horas pico, las rutas más utilizadas, los patrones de viaje por usuario, el tiempo promedio entre estaciones y la detección de rutas críticas, usando funciones puras, colecciones inmutables, lambdas y Streams (Urma et al., 2019; Oracle, s. f.).

El informe describe el diseño de la solución, explica cómo se aplicó cada concepto de la programación funcional, incluye el código fuente y la evidencia de ejecución, y entrega los enlaces del repositorio donde se aloja el proyecto.

Objetivos

Objetivo general

Aplicar conceptos de programación funcional en Java para desarrollar un módulo que procese de manera declarativa grandes volúmenes de datos de transporte urbano y alojar la solución en un repositorio de GitHub.

Objetivos específicos

1. Implementar las seis operaciones de análisis solicitadas (afluencia, horas pico, rutas más utilizadas, patrones de viaje, tiempo promedio entre estaciones y rutas críticas) mediante pipelines de Streams.
2. Aplicar funciones puras, inmutabilidad, funciones de orden superior y lambdas, evitando efectos secundarios y estado compartido mutable.
3. Verificar la corrección de la solución con datos de prueba de resultado conocido y comparar su ejecución secuencial y paralela sobre un conjunto grande de datos simulados.
4. Crear el repositorio en GitHub con el código fuente, el informe y la documentación necesaria para reproducir la ejecución.

Desarrollo

Análisis del caso

Cada registro de transporte contiene el identificador del usuario, la ruta, la estación, la acción («entrada» o «salida») y la marca de tiempo. El módulo debe resolver las tareas de la Tabla 1 cumpliendo las consideraciones técnicas del caso: funciones puras, listas originales sin modificar, lambdas y Streams, enfoque declarativo, posibilidad de paralelizar y ausencia de efectos secundarios.

Tabla 1

Tarea	Descripción	Método
a)	Afluencia por estación: usuarios que ingresan a cada estación en el día.	`afluenciaPorEstacion`
b)	Horas pico: ingresos agrupados por hora del día.	`horasPico`
c)	Rutas más utilizadas: rutas con mayor volumen de uso.	`rutasMasUtilizadas`
d)	Patrones de viaje: estaciones visitadas por cada usuario, en orden.	`patronesDeViaje`
e)	Tiempo promedio entre estaciones consecutivas.	`tiemposPromedioEntreEstaciones`
f)	Detección de sobrecarga: rutas que superan un umbral de ocupación.	`estadoDeRutas`

Diseño de la solución

El proyecto se organiza en tres paquetes con responsabilidades separadas: el modelo de datos inmutable, el generador de datos simulados y el analizador funcional. La clase `Main` ejecuta la verificación, la simulación y la presentación de resultados. La Tabla 2 resume la estructura.

Tabla 2

Clase o paquete	Responsabilidad
`modelo` (Registro, Accion, Conteo, Tramo, TiempoPromedio, EstadoRuta)	Tipos de datos inmutables (records y enum). `Registro` representa un evento de transporte.

Clase o paquete	Responsabilidad
`datos.GeneradorRegistros`	Simula registros de forma determinista (semilla fija), con más tráfico en horas pico. Devuelve una lista inmutable.
`analisis.AnalizadorTransporte`	Contiene las seis operaciones funcionales y las funciones de orden superior reutilizables.
`VerificacionFuncional`	Pruebas con 12 registros cuyo resultado se calculó a mano, sin frameworks externos.
`Main`	Orquesta la ejecución: verificación, simulación, resultados y comparación secuencial/paralelo.

Las operaciones reciben la lista de registros y devuelven resultados nuevos; nunca modifican su entrada. Los datos se simulan con una semilla fija, de modo que la ejecución es repetible y los resultados se pueden comparar entre ejecuciones y entre los modos secuencial y paralelo.

Documentación de los conceptos funcionales

Funciones puras

Una función pura devuelve siempre el mismo resultado para los mismos argumentos y no produce efectos secundarios: no modifica sus parámetros, no escribe en variables externas y no imprime ni lee datos (Urma et al., 2019). Todos los métodos de `AnalizadorTransporte` cumplen esa condición. Por ejemplo, `afluenciaPorEstacion` solo depende de la lista recibida y construye un resultado nuevo:

```
public List<Conteo<String>> afluenciaPorEstacion(List<Registro> registros) {
    Map<String, Long> porEstacion =
        contarPor(registros, Registro::esEntrada, Registro::estacion);
    return ranking(porEstacion, Integer.MAX_VALUE);
}
```

Al no existir estado compartido, tampoco hay condiciones de carrera: el problema de conteos incorrectos del sistema actual desaparece. Las impresiones en consola se concentran en `Main`, fuera del núcleo funcional.

Inmutabilidad

La inmutabilidad significa que un objeto no cambia después de ser creado (Bloch, 2018). Se aplicó en tres niveles. Primero, `Registro` es un `record`, cuyos campos son finales y no tiene métodos que los modifiquen. Segundo, las colecciones que devuelve el generador y el analizador son inmutables, creadas con `toList()`, `List.copyOf` y `Map.copyOf`. Tercero, ninguna operación altera la lista original: el programa lo comprueba intentando añadir un elemento, lo que lanza `UnsupportedOperationException`, y verificando que la lista conserva su tamaño y contenido después del análisis.

```
public record Registro(String idUsuario, String ruta, String estacion,
                       Accion accion, LocalDateTime timestamp) { ... }

return flujo(registros)
        .collect(Collectors.collectingAndThen(
                Collectors.groupingBy(clasificador, Collectors.counting()),
                Map::copyOf));   // el mapa devuelto no se puede modificar
```

Funciones de orden superior

Una función de orden superior recibe funciones como parámetros o las devuelve. El método `contarPor` recibe un `Predicate` (qué registros se cuentan) y una `Function` (por qué clave se agrupan), y por eso resuelve las tareas a), b) y c) sin repetir código. `estadoDeRutas` recibe un `LongPredicate` con el criterio de sobrecarga, de modo que el umbral se define fuera del método. `Main` usa además `Supplier` para ejecutar la misma operación en modo secuencial y paralelo.

```
public <K> Map<K, Long> contarPor(List<Registro> registros,
                                  Predicate<Registro> filtro,
                                  Function<Registro, K> clasificador) { ... }

// Tres tareas distintas reutilizando la misma función:
contarPor(registros, Registro::esEntrada, Registro::estacion);  // a)
contarPor(registros, Registro::esEntrada, Registro::hora);      // b)
contarPor(registros, Registro::esEntrada, Registro::ruta);      // c)
```

Lambdas y referencias a métodos

Una lambda es una función anónima escrita en línea, y una referencia a método (`Registro::estacion`) es su forma abreviada cuando la lambda solo llama a un método existente. El proyecto usa ambas. Un ejemplo es el criterio de ruta crítica, donde el umbral se captura en la lambda `ocupacion -> ocupacion > umbral`, y otro es el comparador que ordena los tramos por número de muestras.

```
secuencial.estadoDeRutas(registros, ocupacion -> ocupacion > umbral);

.sorted((a, b) -> Long.compare(b.muestras(), a.muestras()))
```

Streams

Un Stream es una secuencia de elementos sobre la que se encadenan operaciones declarativas, sin ciclos ni variables acumuladoras, y que no modifica su fuente (Oracle, s. f.). La Tabla 3 muestra el pipeline de cada tarea.

Tabla 3

Tarea	Pipeline (operaciones principales)
a) Afluencia	`filter` (solo entradas) → `groupingBy(estación, counting)` → `sorted` → `toList`
b) Horas pico	`filter` → `groupingBy(hora, counting)` → `sorted` descendente → `limit(n)`
c) Rutas	`filter` → `groupingBy(ruta, counting)` → `sorted` → `limit(n)`
d) Patrones	`groupingBy(usuario, mapping(estación, toList))` → `collectingAndThen(Map::copyOf)`
e) Tiempos	`groupingBy(usuario)` → `flatMap` (tramos consecutivos con `IntStream.range`) → `filter` → `groupingBy(tramo, teeing(averagingLong, counting))`
f) Sobrecarga	conteo por ruta → `map` (aplica el `LongPredicate`) → `sorted` → `toList`

Paralelización

Como las funciones son puras y no comparten estado mutable, el mismo código se puede ejecutar con `parallelStream()`. El analizador recibe un indicador `paralelo` fijado al

construirlo, y `Main` compara ambos modos comprobando que los resultados sean iguales. Una ejecución paralela no siempre es más rápida: depende del número de núcleos y del costo de dividir y combinar los datos, por lo que los tiempos se presentan como evidencia y no como garantía de mejora.

Código fuente

A continuación se presenta el código fuente completo del proyecto, escrito en Java 17 sin librerías externas.

modelo/Accion.java

```
package co.edu.iudigital.tecnomovil.modelo;

/** Tipo de evento que registra un validador o sensor: "entrada" o "salida". */
public enum Accion {
    ENTRADA("entrada"),
    SALIDA("salida");

    private final String texto;

    Accion(String texto) {
        this.texto = texto;
    }

    public String texto() {
        return texto;
    }
}
```

modelo/Registro.java

```
package co.edu.iudigital.tecnomovil.modelo;

import java.time.LocalDateTime;
import java.util.Objects;

/**
 * Registro de un evento de transporte (requerimiento 3.1).
 *
 * Es un {@code record}: todos sus campos son finales, no tiene setters y, por tanto,
 * una vez creado NUNCA cambia (objeto inmutable).
 */
public record Registro(String idUsuario, String ruta, String estacion, Accion accion,
LocalDateTime timestamp) {

    public Registro {
        Objects.requireNonNull(idUsuario, "idUsuario");
        Objects.requireNonNull(ruta, "ruta");
        Objects.requireNonNull(estacion, "estacion");
```

```
        Objects.requireNonNull(accion, "accion");
        Objects.requireNonNull(timestamp, "timestamp");
    }

    /** Hora del día (0-23) en la que ocurrió el evento. */
    public int hora() {
        return timestamp.getHour();
    }

    public boolean esEntrada() {
        return accion == Accion.ENTRADA;
    }
}
```

modelo/Conteo.java

```
package co.edu.iudigital.tecnomovil.modelo;

/** Par (clave, total) usado para devolver conteos y rankings. Inmutable. */
public record Conteo<K>(K clave, long total) { }
```

modelo/Tramo.java

```
package co.edu.iudigital.tecnomovil.modelo;

/** Recorrido entre dos eventos consecutivos de un mismo usuario. Inmutable. */
public record Tramo(String origen, String destino, long segundos) {

    public String etiqueta() {
        return origen + " -> " + destino;
    }
}
```

modelo/TiempoPromedio.java

```
package co.edu.iudigital.tecnomovil.modelo;

/** Tiempo promedio (en minutos) de un tramo entre estaciones y número de muestras
usadas. */
public record TiempoPromedio(String tramo, double minutos, long muestras) { }
```

modelo/EstadoRuta.java

```
package co.edu.iudigital.tecnomovil.modelo;

/** Ocupación (simulada) de una ruta y si se marcó como "crítica". */
public record EstadoRuta(String ruta, long ocupacion, boolean critica) { }
```

datos/GeneradorRegistros.java

```
package co.edu.iudigital.tecnomovil.datos;

import co.edu.iudigital.tecnomovil.modelo.Accion;
import co.edu.iudigital.tecnomovil.modelo.Registro;
```

```
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.LocalTime;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;
import java.util.SplittableRandom;
import java.util.stream.IntStream;

/**
 * Genera datos SIMULADOS de validadores y sensores.
 *
 * Es determinista: con los mismos parámetros (usuarios, semilla) produce siempre
exactamente
 * los mismos registros, así que los resultados del análisis se pueden repetir y
comparar.
 * La lista devuelta es inmutable.
 */
public final class GeneradorRegistros {

    public static final List<String> ESTACIONES = List.of(
            "Central", "Norte", "Sur", "Oriente", "Occidente", "Universidad",
            "Aeropuerto", "Terminal", "Hospital", "Parque", "Plaza Mayor",
"Industrial");

    public static final List<String> RUTAS = List.of("R1", "R2", "R3", "R4", "R5",
"R6", "A1", "A2");

    public static final LocalDate DIA = LocalDate.of(2026, 10, 20);

    private GeneradorRegistros() { }

    /** Genera los registros de {@code usuarios} personas durante un día, ordenados
por timestamp. */
    public static List<Registro> generar(int usuarios, long semilla) {
        return IntStream.range(0, usuarios)
                .mapToObj(u -> registrosDeUsuario(u, semilla))
                .flatMap(List::stream)

.sorted(Comparator.comparing(Registro::timestamp).thenComparing(Registro::idUsuario))
                .toList(); // toList() devuelve una lista inmutable
    }

    private static List<Registro> registrosDeUsuario(int numeroUsuario, long semilla)
{
        SplittableRandom azar = new SplittableRandom(semilla + numeroUsuario *
7_919L);
        String idUsuario = String.format("U%06d", numeroUsuario);
        int viajes = 1 + azar.nextInt(3);

        List<Registro> resultado = new ArrayList<>(); // lista local, se devuelve ya
inmutable
        LocalDateTime inicio = LocalDateTime.of(DIA, LocalTime.of(horaInicial(azar),
azar.nextInt(60), azar.nextInt(60)));
        String origen = estacionSesgada(azar);

        for (int i = 0; i < viajes && inicio.toLocalDate().equals(DIA); i++) {
            String destino = destinoDistinto(azar, origen);
            String ruta = RUTAS.get(Math.min(azar.nextInt(RUTAS.size()),
azar.nextInt(RUTAS.size())));
```

```
            LocalDateTime fin = inicio.plusMinutes(8 + azar.nextInt(38));

            resultado.add(new Registro(idUsuario, ruta, origen, Accion.ENTRADA,
inicio));
            resultado.add(new Registro(idUsuario, ruta, destino, Accion.SALIDA, fin));

            origen = destino;
            inicio = fin.plusMinutes(30 + azar.nextInt(150));
        }
        return List.copyOf(resultado);
    }

    /** Más tráfico en la mañana (6-8) y en la tarde (16-19) que en el resto del día.
*/
    private static int horaInicial(SplittableRandom azar) {
        int sorteo = azar.nextInt(100);
        if (sorteo < 35) return 6 + azar.nextInt(3);
        if (sorteo < 65) return 16 + azar.nextInt(4);
        return 5 + azar.nextInt(17);
    }

    /** Algunas estaciones (las primeras de la lista) reciben más usuarios que otras.
*/
    private static String estacionSesgada(SplittableRandom azar) {
        int n = ESTACIONES.size();
        return ESTACIONES.get(Math.min(azar.nextInt(n), azar.nextInt(n)));
    }

    private static String destinoDistinto(SplittableRandom azar, String origen) {
        String destino = estacionSesgada(azar);
        while (destino.equals(origen)) {
            destino = ESTACIONES.get(azar.nextInt(ESTACIONES.size()));
        }
        return destino;
    }
}
```

analisis/AnalizadorTransporte.java

```
package co.edu.iudigital.tecnomovil.analisis;

import co.edu.iudigital.tecnomovil.modelo.Conteo;
import co.edu.iudigital.tecnomovil.modelo.EstadoRuta;
import co.edu.iudigital.tecnomovil.modelo.Registro;
import co.edu.iudigital.tecnomovil.modelo.TiempoPromedio;
import co.edu.iudigital.tecnomovil.modelo.Tramo;

import java.util.Collection;
import java.util.Comparator;
import java.util.List;
import java.util.Map;
import java.util.function.Function;
import java.util.function.LongPredicate;
import java.util.function.Predicate;
import java.util.stream.Collectors;
import java.util.stream.IntStream;
import java.util.stream.Stream;

/**
```

```
 * Operaciones funcionales del módulo de procesamiento y agregación de datos diarios
 * (requerimiento 3.2, tareas a-f).
 *
 * Reglas de diseño:
 *  - Cada método es una FUNCIÓN PURA: el resultado depende solo de los argumentos, no
lee ni
 *    escribe estado externo y no modifica la lista recibida.
 *  - Todos los resultados son colecciones INMUTABLES (List.copyOf, Map.copyOf,
toList()).
 *  - Todo se resuelve con pipelines de Streams y lambdas, sin ciclos ni variables
acumuladoras.
 *  - La única "configuración" es el campo final {@code paralelo}: con true se usa
parallelStream().
 *    Como no hay estado compartido mutable, el resultado es el mismo en ambos modos.
 */
public final class AnalizadorTransporte {

    private final boolean paralelo;

    public AnalizadorTransporte(boolean paralelo) {
        this.paralelo = paralelo;
    }

    /** Crea el flujo (secuencial o paralelo) sin modificar la colección original. */
    private <T> Stream<T> flujo(Collection<T> datos) {
        return paralelo ? datos.parallelStream() : datos.stream();
    }

    // ------------------------------------------------------------------
    // Funciones de orden superior reutilizables
    // ------------------------------------------------------------------

    /**
     * Cuenta registros agrupándolos por una clave. Recibe DOS funciones como
parámetro:
     * un {@link Predicate} (qué registros cuentan) y una {@link Function} (por qué se
agrupan).
     */
    public <K> Map<K, Long> contarPor(List<Registro> registros,
                                      Predicate<Registro> filtro,
                                      Function<Registro, K> clasificador) {
        return flujo(registros)
                .filter(filtro)
                .collect(Collectors.collectingAndThen(
                        Collectors.groupingBy(clasificador, Collectors.counting()),
                        Map::copyOf));
    }

    /** Ordena un mapa de conteos de mayor a menor (desempate por clave) y toma los
primeros n. */
    public static <K extends Comparable<K>> List<Conteo<K>> ranking(Map<K, Long>
conteos, int n) {
        return conteos.entrySet().stream()
                .map(e -> new Conteo<>(e.getKey(), e.getValue()))
                .sorted(Comparator.<Conteo<K>>comparingLong(Conteo::total).reversed()
                        .thenComparing(Conteo::clave))
                .limit(n)
                .toList();
    }

    // ------------------------------------------------------------------
```

```
    // a) Afluencia por estación
    // ------------------------------------------------------------------

    /** Cuántos usuarios ingresan a cada estación durante el día (de mayor a menor).
*/
    public List<Conteo<String>> afluenciaPorEstacion(List<Registro> registros) {
        Map<String, Long> porEstacion = contarPor(registros, Registro::esEntrada,
Registro::estacion);
        return ranking(porEstacion, Integer.MAX_VALUE);
    }

    // ------------------------------------------------------------------
    // b) Horas pico
    // ------------------------------------------------------------------

    /** Las {@code n} horas del día con más ingresos de pasajeros. */
    public List<Conteo<Integer>> horasPico(List<Registro> registros, int n) {
        Map<Integer, Long> porHora = contarPor(registros, Registro::esEntrada,
Registro::hora);
        return ranking(porHora, n);
    }

    // ------------------------------------------------------------------
    // c) Rutas más utilizadas
    // ------------------------------------------------------------------

    /** Las {@code n} rutas con mayor volumen de uso (ingresos). */
    public List<Conteo<String>> rutasMasUtilizadas(List<Registro> registros, int n) {
        Map<String, Long> porRuta = contarPor(registros, Registro::esEntrada,
Registro::ruta);
        return ranking(porRuta, n);
    }

    // ------------------------------------------------------------------
    // d) Patrones de viaje por usuario
    // ------------------------------------------------------------------

    /**
     * Para cada usuario, la lista de estaciones visitadas en el orden en que aparecen
     * en los registros. groupingBy conserva el orden de encuentro de una lista
ordenada,
     * también en modo paralelo.
     */
    public Map<String, List<String>> patronesDeViaje(List<Registro> registros) {
        return flujo(registros)
                .collect(Collectors.collectingAndThen(
                        Collectors.groupingBy(
                                Registro::idUsuario,
                                Collectors.collectingAndThen(
                                        Collectors.mapping(Registro::estacion,
Collectors.toList()),
                                        List::copyOf)),
                        Map::copyOf));
    }

    // ------------------------------------------------------------------
    // e) Tiempo promedio entre estaciones
    // ------------------------------------------------------------------

    private record Resumen(double segundosPromedio, long muestras) { }
```

```
    /**
     * Tiempo promedio que tardan los usuarios entre estaciones consecutivas.
     * Función auxiliar pura: convierte los registros de UN usuario en tramos entre
     * eventos consecutivos (ordenados por tiempo) que ocurren en estaciones
distintas.
     */
    private static Stream<Tramo> tramosConsecutivos(List<Registro> delUsuario) {
        List<Registro> ordenados = delUsuario.stream()
                .sorted(Comparator.comparing(Registro::timestamp))
                .toList();
        return IntStream.range(0, ordenados.size() - 1)
                .mapToObj(i -> {
                    Registro desde = ordenados.get(i);
                    Registro hasta = ordenados.get(i + 1);
                    long segundos = java.time.Duration.between(desde.timestamp(),
hasta.timestamp()).getSeconds();
                    return new Tramo(desde.estacion(), hasta.estacion(), segundos);
                })
                .filter(t -> !t.origen().equals(t.destino()) && t.segundos() >= 0);
    }

    public List<TiempoPromedio> tiemposPromedioEntreEstaciones(List<Registro>
registros) {
        Map<String, Resumen> porTramo = flujo(registros)
                .collect(Collectors.groupingBy(Registro::idUsuario))          //
Map<usuario, registros>
                .values().stream()
                .flatMap(AnalizadorTransporte::tramosConsecutivos)             //
Stream<Tramo>
                .collect(Collectors.groupingBy(
                        Tramo::etiqueta,
                        Collectors.teeing(
                                Collectors.averagingLong(Tramo::segundos),
                                Collectors.counting(),
                                Resumen::new)));

        return porTramo.entrySet().stream()
                .map(e -> new TiempoPromedio(e.getKey(),
e.getValue().segundosPromedio() / 60.0, e.getValue().muestras()))
                .sorted(Comparator.comparing(TiempoPromedio::tramo))
                .toList();
    }

    // ------------------------------------------------------------------
    // f) Detección de sobrecarga en rutas
    // ------------------------------------------------------------------

    /**
     * Marca como crítica una ruta cuando su ocupación (simulada como el número de
ingresos
     * del día) cumple el criterio recibido. El criterio es una lambda: {@code
ocupacion -> ocupacion > umbral}.
     */
    public List<EstadoRuta> estadoDeRutas(List<Registro> registros, LongPredicate
esCritica) {
        return contarPor(registros, Registro::esEntrada,
Registro::ruta).entrySet().stream()
                .map(e -> new EstadoRuta(e.getKey(), e.getValue(),
esCritica.test(e.getValue())))
                .sorted(Comparator.comparingLong(EstadoRuta::ocupacion).reversed()
```

```
                        .thenComparing(EstadoRuta::ruta))
                .toList();
    }
}
```

VerificacionFuncional.java

```
package co.edu.iudigital.tecnomovil;

import co.edu.iudigital.tecnomovil.analisis.AnalizadorTransporte;
import co.edu.iudigital.tecnomovil.modelo.Accion;
import co.edu.iudigital.tecnomovil.modelo.Conteo;
import co.edu.iudigital.tecnomovil.modelo.EstadoRuta;
import co.edu.iudigital.tecnomovil.modelo.Registro;
import co.edu.iudigital.tecnomovil.modelo.TiempoPromedio;

import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import java.util.stream.Stream;

/**
 * Comprobaciones con un conjunto PEQUEÑO de datos cuyo resultado se calculó a mano.
 * No usa frameworks externos: cada prueba compara el valor esperado con el obtenido.
 */
public final class VerificacionFuncional {

    public record Resultado(String nombre, boolean correcto, String esperado, String
obtenido) { }

    private VerificacionFuncional() { }

    private static Registro reg(String usuario, String ruta, String estacion, Accion
accion, int hora, int minuto) {
        return new Registro(usuario, ruta, estacion, accion, LocalDateTime.of(2026,
10, 20, hora, minuto));
    }

    /** Conjunto de prueba (12 registros de 5 usuarios) con respuesta conocida. */
    public static List<Registro> datosDePrueba() {
        return List.of(
                reg("U1", "R1", "Central", Accion.ENTRADA, 7, 0),
                reg("U1", "R1", "Norte", Accion.SALIDA, 7, 20),
                reg("U2", "R1", "Central", Accion.ENTRADA, 7, 10),
                reg("U3", "R2", "Norte", Accion.ENTRADA, 7, 30),
                reg("U2", "R1", "Norte", Accion.SALIDA, 7, 40),
                reg("U5", "R1", "Central", Accion.ENTRADA, 7, 45),
                reg("U3", "R2", "Sur", Accion.SALIDA, 7, 55),
                reg("U5", "R1", "Norte", Accion.SALIDA, 8, 5),
                reg("U1", "R2", "Norte", Accion.ENTRADA, 17, 0),
                reg("U4", "R3", "Sur", Accion.ENTRADA, 17, 20),
                reg("U1", "R2", "Sur", Accion.SALIDA, 17, 30),
                reg("U4", "R3", "Central", Accion.SALIDA, 17, 50));
    }

    private static Resultado comprobar(String nombre, Object esperado, Object
obtenido) {
```

```
        return new Resultado(nombre, esperado.equals(obtenido),
String.valueOf(esperado), String.valueOf(obtenido));
    }

    private static List<String> textos(List<Conteo<String>> conteos) {
        return conteos.stream().map(c -> c.clave() + "=" + c.total()).toList();
    }

    public static List<Resultado> ejecutar(boolean paralelo) {
        AnalizadorTransporte analizador = new AnalizadorTransporte(paralelo);
        List<Registro> datos = datosDePrueba();
        List<Registro> copiaAntes = new ArrayList<>(datos); // foto de la lista antes
de analizar

        Map<String, List<String>> patrones = analizador.patronesDeViaje(datos);
        List<TiempoPromedio> tiempos =
analizador.tiemposPromedioEntreEstaciones(datos);
        List<EstadoRuta> estados = analizador.estadoDeRutas(datos, ocupacion ->
ocupacion > 2);

        return Stream.of(
                comprobar("a) Afluencia por estación",
                        List.of("Central=3", "Norte=2", "Sur=1"),
                        textos(analizador.afluenciaPorEstacion(datos))),
                comprobar("b) Hora pico (hora=ingresos)",
                        List.of(new Conteo<>(7, 4L), new Conteo<>(17, 2L)),
                        analizador.horasPico(datos, 2)),
                comprobar("c) Rutas más utilizadas",
                        List.of("R1=3", "R2=2", "R3=1"),
                        textos(analizador.rutasMasUtilizadas(datos, 3))),
                comprobar("d) Patrón de viaje de U1",
                        List.of("Central", "Norte", "Norte", "Sur"),
                        patrones.get("U1")),
                comprobar("d) Patrón de viaje de U4",
                        List.of("Sur", "Central"),
                        patrones.get("U4")),
                comprobar("e) Central -> Norte (minutos, muestras)",
                        "23.3 min (3 muestras)",
                        tiempos.stream().filter(t -> t.tramo().equals("Central ->
Norte"))
                                .map(t -> String.format(java.util.Locale.US, "%.1f min
(%d muestras)", t.minutos(), t.muestras()))
                                .findFirst().orElse("sin datos")),
                comprobar("e) Norte -> Sur (minutos, muestras)",
                        "27.5 min (2 muestras)",
                        tiempos.stream().filter(t -> t.tramo().equals("Norte -> Sur"))
                                .map(t -> String.format(java.util.Locale.US, "%.1f min
(%d muestras)", t.minutos(), t.muestras()))
                                .findFirst().orElse("sin datos")),
                comprobar("f) Rutas críticas (umbral > 2)",
                        List.of("R1"),

estados.stream().filter(EstadoRuta::critica).map(EstadoRuta::ruta).toList()),
                comprobar("Inmutabilidad: la lista original no cambió tras el
análisis",
                        true,
                        datos.equals(copiaAntes))
        ).toList();
    }
}
```

Main.java

```
package co.edu.iudigital.tecnomovil;

import co.edu.iudigital.tecnomovil.analisis.AnalizadorTransporte;
import co.edu.iudigital.tecnomovil.datos.GeneradorRegistros;
import co.edu.iudigital.tecnomovil.modelo.Accion;
import co.edu.iudigital.tecnomovil.modelo.Conteo;
import co.edu.iudigital.tecnomovil.modelo.EstadoRuta;
import co.edu.iudigital.tecnomovil.modelo.Registro;
import co.edu.iudigital.tecnomovil.modelo.TiempoPromedio;

import java.io.FileDescriptor;
import java.io.FileOutputStream;
import java.io.PrintStream;
import java.nio.charset.StandardCharsets;
import java.time.LocalDateTime;
import java.util.List;
import java.util.Locale;
import java.util.Map;
import java.util.function.Supplier;

/**
 * TecnoMovil Data: módulo funcional de procesamiento y agregación de datos diarios de
transporte.
 *
 * Uso:  java -cp out co.edu.iudigital.tecnomovil.Main [usuarios]
 * Por defecto se simulan 250.000 usuarios (≈ 1 millón de registros).
 */
public final class Main {

    private Main() { }

    public static void main(String[] args) {
        // Salida en UTF-8 para que las tildes se vean bien en la consola
        System.setOut(new PrintStream(new FileOutputStream(FileDescriptor.out), true,
StandardCharsets.UTF_8));

        int usuarios = args.length > 0 ? Integer.parseInt(args[0]) : 250_000;

        titulo("1. VERIFICACIÓN CON DATOS DE PRUEBA (resultado calculado a mano)");
        List<VerificacionFuncional.Resultado> pruebas =
VerificacionFuncional.ejecutar(false);
        pruebas.forEach(p -> System.out.printf("  [%s] %s%n", p.correcto() ? "OK" :
"FALLO", p.nombre()));
        long correctas =
pruebas.stream().filter(VerificacionFuncional.Resultado::correcto).count();
        System.out.printf("  Resultado: %d de %d pruebas correctas%n", correctas,
pruebas.size());

        titulo("2. SIMULACIÓN DE DATOS");
        long t0 = System.nanoTime();
        List<Registro> registros = GeneradorRegistros.generar(usuarios, 2026L);
        System.out.printf(Locale.US, "  Usuarios simulados : %,d%n", usuarios);
        System.out.printf(Locale.US, "  Registros generados: %,d (%d ms)%n",
registros.size(), ms(t0));

        titulo("3. INMUTABILIDAD");
        try {
```

```
            registros.add(new Registro("X", "R1", "Central", Accion.ENTRADA,
LocalDateTime.now()));
            System.out.println("  ERROR: la lista se pudo modificar");
        } catch (UnsupportedOperationException e) {
            System.out.println("  La lista original es inmutable: add() lanzó
UnsupportedOperationException");
        }

        AnalizadorTransporte secuencial = new AnalizadorTransporte(false);
        long entradas = registros.stream().filter(Registro::esEntrada).count();
        long umbral = Math.round(entradas / (double) GeneradorRegistros.RUTAS.size() *
1.25);

        titulo("4. RESULTADOS (procesamiento secuencial)");

        subtitulo("a) Afluencia por estación (ingresos del día)");
        secuencial.afluenciaPorEstacion(registros).forEach(c ->
System.out.printf(Locale.US, "  %-12s %,10d%n", c.clave(), c.total()));

        subtitulo("b) Horas pico (3 horas con más ingresos)");
        secuencial.horasPico(registros, 3).forEach(c -> System.out.printf(Locale.US, "
%02d:00 - %02d:59 %,10d%n", c.clave(), c.clave(), c.total()));

        subtitulo("c) Rutas más utilizadas (top 5)");
        secuencial.rutasMasUtilizadas(registros, 5).forEach(c ->
System.out.printf(Locale.US, "  %-4s %,10d%n", c.clave(), c.total()));

        subtitulo("d) Patrones de viaje (primeros 3 usuarios)");
        secuencial.patronesDeViaje(registros).entrySet().stream()
                .sorted(Map.Entry.comparingByKey())
                .limit(3)
                .forEach(e -> System.out.printf("  %s: %s%n", e.getKey(),
String.join(" > ", e.getValue())));

        subtitulo("e) Tiempo promedio entre estaciones (5 tramos con más muestras)");
        secuencial.tiemposPromedioEntreEstaciones(registros).stream()
                .sorted((a, b) -> Long.compare(b.muestras(), a.muestras()))
                .limit(5)
                .forEach(t -> System.out.printf(Locale.US, "  %-24s %6.1f min  (%,d
muestras)%n", t.tramo(), t.minutos(), t.muestras()));

        subtitulo("f) Estado de las rutas (umbral crítico: más de " +
String.format(Locale.US, "%,d", umbral) + " ingresos)");
        secuencial.estadoDeRutas(registros, ocupacion -> ocupacion > umbral).forEach(r
->
                System.out.printf(Locale.US, "  %-4s %,10d  %s%n", r.ruta(),
r.ocupacion(), r.critica() ? "CRÍTICA" : "normal"));

        titulo("5. SECUENCIAL vs PARALELO (parallelStream)");
        AnalizadorTransporte paralelo = new AnalizadorTransporte(true);
        compararModos("a) Afluencia por estación",
                () -> secuencial.afluenciaPorEstacion(registros), () ->
paralelo.afluenciaPorEstacion(registros));
        compararModos("b) Horas pico",
                () -> secuencial.horasPico(registros, 3), () ->
paralelo.horasPico(registros, 3));
        compararModos("c) Rutas más utilizadas",
                () -> secuencial.rutasMasUtilizadas(registros, 5), () ->
paralelo.rutasMasUtilizadas(registros, 5));
        compararModos("d) Patrones de viaje",
```

```
                () -> secuencial.patronesDeViaje(registros), () ->
paralelo.patronesDeViaje(registros));
        compararModos("e) Tiempos promedio",
                () -> secuencial.tiemposPromedioEntreEstaciones(registros), () ->
paralelo.tiemposPromedioEntreEstaciones(registros));
        compararModos("f) Estado de rutas",
                () -> secuencial.estadoDeRutas(registros, o -> o > umbral), () ->
paralelo.estadoDeRutas(registros, o -> o > umbral));

        System.out.println();
        System.out.printf("  La lista original sigue con %,d registros (no se
modificó).%n", registros.size());
    }

    /** Ejecuta la misma operación en modo secuencial y paralelo y verifica que den el
mismo resultado. */
    private static <T> void compararModos(String nombre, Supplier<T> secuencial,
Supplier<T> paralelo) {
        long t1 = System.nanoTime();
        T r1 = secuencial.get();
        long msSecuencial = ms(t1);
        long t2 = System.nanoTime();
        T r2 = paralelo.get();
        long msParalelo = ms(t2);
        System.out.printf("  %-28s secuencial %5d ms | paralelo %5d ms | mismo
resultado: %s%n",
                nombre, msSecuencial, msParalelo, r1.equals(r2) ? "SÍ" : "NO");
    }

    private static long ms(long desdeNanos) {
        return (System.nanoTime() - desdeNanos) / 1_000_000;
    }

    private static void titulo(String texto) {
        System.out.println();
        System.out.println("=".repeat(78));
        System.out.println(texto);
        System.out.println("=".repeat(78));
    }

    private static void subtitulo(String texto) {
        System.out.println();
        System.out.println(" " + texto);
    }
}
```

Evidencia de ejecución

Las capturas de las Figuras 1 a 3 muestran la compilación y ejecución del proyecto en el equipo de la autora, con 250.000 usuarios simulados que generan 980.006 registros. La Figura 1 presenta la verificación con datos de prueba (9 de 9 pruebas correctas), la simulación y la comprobación de inmutabilidad. La Figura 2 muestra los resultados de las tareas a) a d), y la Figura 3 los de las tareas e) y f) junto con la comparación entre ejecución secuencial y paralela.

Figura 1

Compilación, verificación con datos de prueba, simulación e inmutabilidad

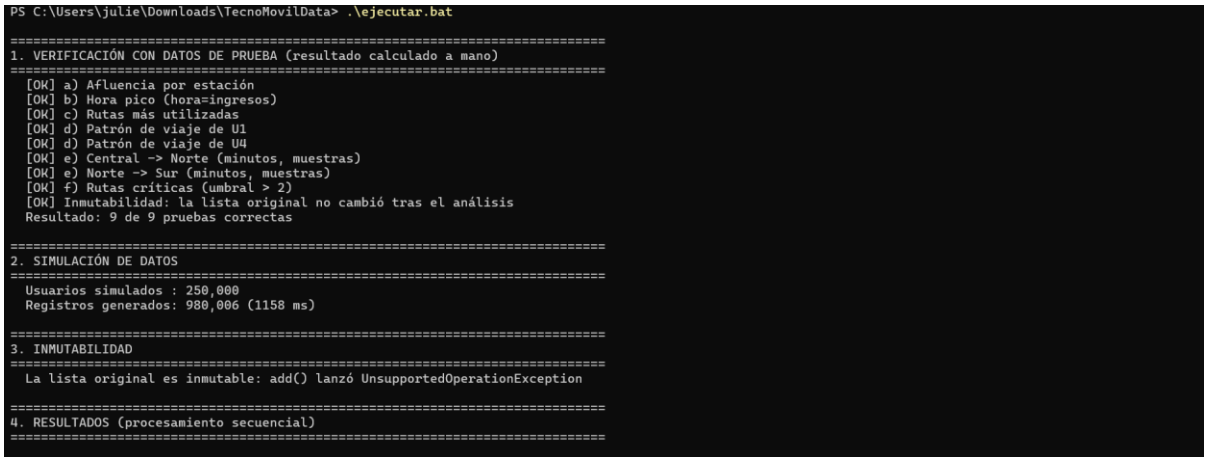


Figura 2

Resultados de las tareas a) a d): afluencia, horas pico, rutas y patrones de viaje

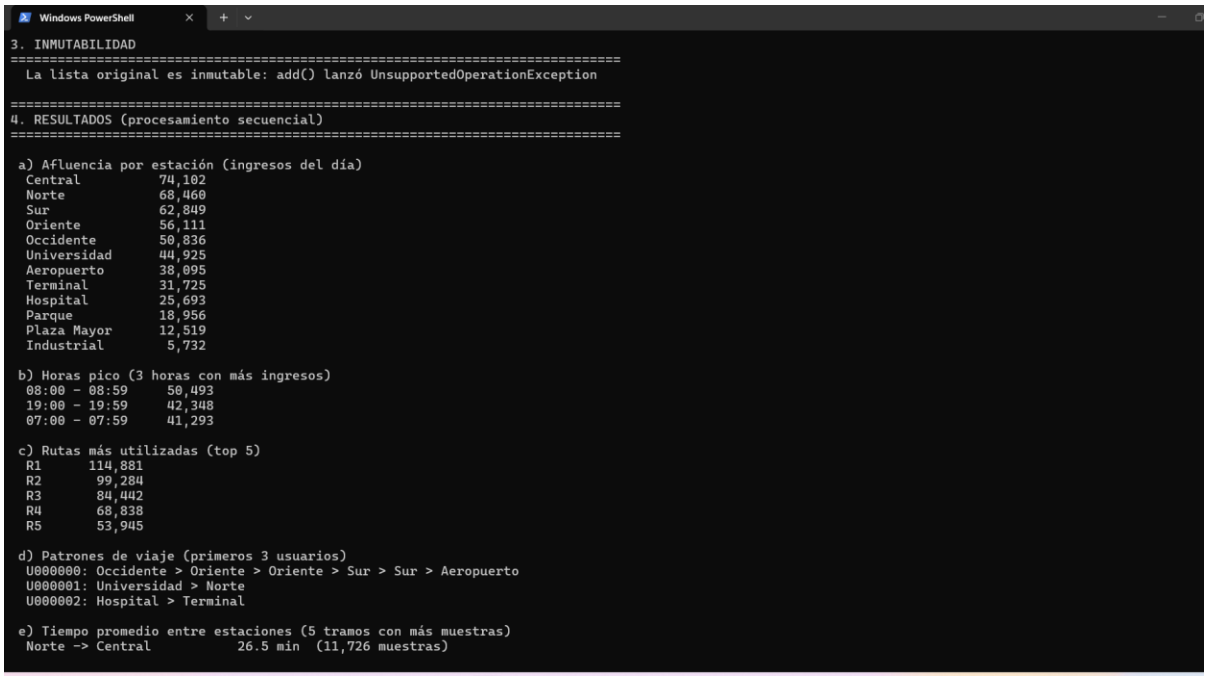
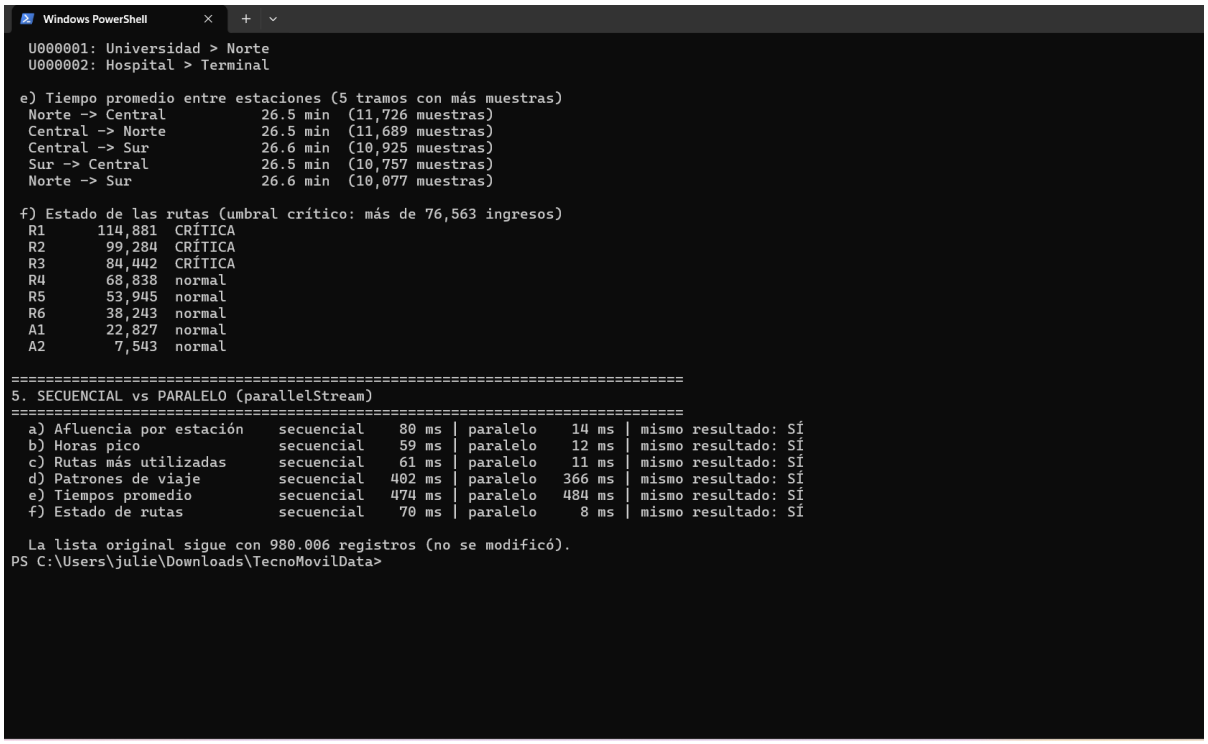


Figura 3

Resultados de las tareas e) y f) y comparación entre ejecución secuencial y paralela



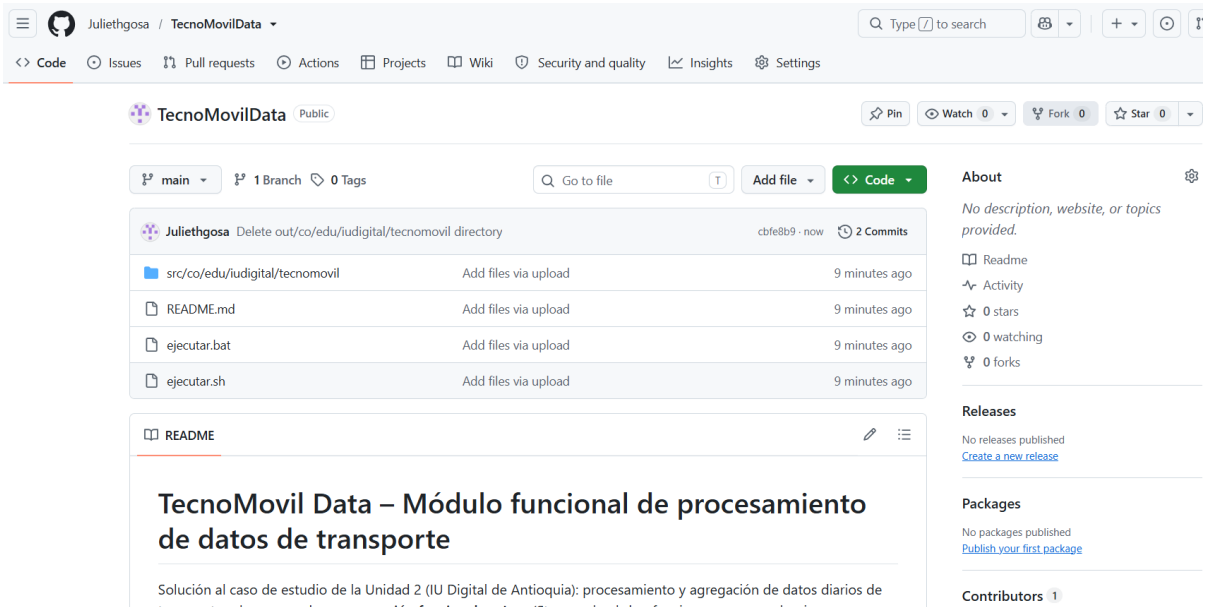
En la comparación de la Figura 3, las operaciones de conteo (tareas a, b, c y f) fueron entre 5 y 9 veces más rápidas con `parallelStream()`, mientras que los patrones de viaje y los tiempos promedio tardaron casi lo mismo en ambos modos. En los seis casos el resultado fue idéntico al del modo secuencial.

Repositorio y archivo ZIP

El proyecto se aloja en un repositorio de GitHub, en la rama principal (main), con el código fuente, el archivo `README.md` con las instrucciones de ejecución, los scripts `ejecutar.sh` y `ejecutar.bat`, y este informe en la carpeta `docs`. Con el mismo contenido se generó el archivo ZIP `TecnoMovilData.zip`, que acompaña la entrega.

Figura 4

Repositorio en GitHub, rama principal



Conclusiones

Se implementaron las seis operaciones de análisis solicitadas con pipelines de Streams, sin ciclos ni variables acumuladoras. Con esto se cumple el primer objetivo, y las nueve pruebas con datos de resultado conocido confirman que los cálculos son correctos.

Se aplicaron funciones puras, colecciones inmutables, funciones de orden superior y lambdas. Al eliminar el estado compartido mutable, se elimina también la causa de las condiciones de carrera que afectaban los conteos del sistema actual, y el programa lo evidencia obteniendo los mismos resultados en ejecución secuencial y paralela.

La ejecución sobre 980.006 registros simulados mostró que el enfoque declarativo permite procesar grandes volúmenes con un código breve y legible. La versión paralela fue entre 5 y 9 veces más rápida en las operaciones de conteo, pero no mejoró los patrones de viaje ni los tiempos promedio, lo que confirma que conviene medir antes de decidir si paralelizar.

Finalmente, el repositorio de GitHub reúne el código, la documentación y el informe, lo que permite reproducir la solución en cualquier equipo con JDK 17 o superior.

Enlaces de los repositorios

María Julieth Gómez Sarrazola: https://github.com/Juliethgosa/TecnoMovilData

Referencias

Bloch, J. (2018). *Effective Java* (3.ª ed.). Addison-Wesley.

Oracle. (s. f.). *Package java.util.stream*. Java Platform, Standard Edition 17 API Specification. https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/stream/package-summary.html

Urma, R.-G., Fusco, M., & Mycroft, A. (2019). *Modern Java in action: Lambdas, streams, functional and reactive programming* (2.ª ed.). Manning.